



CHAPTER 4

텍스트의 전처리, 어휘, 구문 그리고 의미 분석

텍스트의 구조와 의미는 어떻게 분석하는가?

학습 목표

- ① 텍스트 전처리
 - 비정형 데이터의 특성과 오류 유형을 설명할 수 있다
 - 정규표현식 · 유니코드 정규화 · 토큰화를 적용할 수 있다
 - 한국어 띄어쓰기 교정 기법 3가지와 맞춤법 교정 기법 2가지를 비교할 수 있다
- ② 어휘 분석
 - 형태소와 품사의 개념을 정의할 수 있다
 - 한국어 형태소 분석기의 차이를 실험으로 비교할 수 있다
 - HMM 기반 품사 태깅의 원리를 수식으로 설명할 수 있다
- ③ 구문 분석
 - 구구조 문법과 의존 문법의 차이를 설명할 수 있다
 - 구문 중의성이 왜 발생하는지 예로 들 수 있다 / 전이 기반·그래프 기반 파싱을 구분할 수 있다
- ④ 의미 분석
 - 어휘적 중의성과 그 해소 기법을 설명할 수 있다
 - 의미역(semantic role)의 종류를 분류하고, 분포 가설에 기반한 의미 표현을 설명할 수 있다

자연어처리의 분석 계층

낮은 계층의 오류는 상위 계층으로 그대로 전파된다 (error propagation)

- STEP 1 전처리 문서 → 깨끗한 문자열 (노이즈 · 인코딩 · 띄어쓰기)
- STEP 2 어휘 분석 문자열 → 형태소 · 품사 (단어 수준의 이해)
- STEP 3 구문 분석 단어 → 문법 구조 (누가 무엇을 수식하는가)
- STEP 4 의미 분석 구조 → 의미 표현 (무엇을 뜻하는가)

파이프라인 구조

- 각 계층은 독립적인 과제가 아니라, 하위 계층의 출력이 상위 계층의 입력이 되는 구조를 이룬다
- 예) 띄어쓰기가 잘못되면 형태소 분석이 실패하고 → 품사 태깅이 틀리며 → 구문 트리가 어긋나고 → 최종 의미 해석까지 오류가 누적된다
- 따라서 성능 개선의 출발점은 대개 모델이 아니라 전처리에 있다 (Garbage In, Garbage Out)
- 최근의 사전학습 언어모델은 이 계층들을 하나의 신경망 안에서 암묵적으로 처리하지만, 결과를 해석하고 오류를 진단하려면 각 계층의 개념을 알아야 한다

이번 주치의 관점 : 규칙 기반 → 통계 기반 → 딥러닝 기반

- 네 개 절 모두 이 세 가지 접근법이 반복해서 등장한다. 장단점을 비교하며 읽으면 전체 흐름이 하나로 연결된다

목차

Chapter 4는 네 개의 절로 구성된다

4.1 텍스트의 전처리

- 비정형 데이터 내의 오류 / 텍스트 문서의 변환 / 정제 · 정규화 · 토큰화
- 띄어쓰기 교정 방법 / 철자 및 맞춤법 교정 방법 / 서브워드 토큰화

4.2 어휘 분석

- 형태소 분석 / 한국어 형태소 분석기 비교 / 품사와 품사 태그셋
- 품사 태깅 접근법 / 형태소 분석 · 품사 태깅의 활용

4.3 구문 분석

- 구문 분석 개요 / 구구조 문법과 의존 문법 / 구문 중의성
- 구구조 구문 분석 / 의존 구문 분석 / 접근 방법의 장단점

4.4 의미 분석

- 단어 의미 중의성 / 중의성 해소(WSD) 기법 / 의미역 분석 / 의미 표현과 분포 가설

4.1 텍스트의 전처리

비정형 데이터 내의 오류 · 문서 변환 · 정제 · 정규화 · 토큰화 · 띄어쓰기 교정 · 맞춤법 교정 · 서브워드 토큰화

4.1 정형 데이터와 비정형 데이터

텍스트는 대표적인 비정형 데이터이며, 그대로는 계산의 대상이 되지 않는다

정형 데이터 (Structured Data)

- 일정한 규격 · 스키마를 가진 데이터
- 행과 열로 정의되며 각 칸의 의미가 명확함
- 관계형 데이터베이스 · 스프레드시트에 저장
- 예) 고객 ID, 나이, 구매 금액, 센서 측정값 → 정렬 · 집계 · 조인 등 연산이 즉시 가능

비정형 데이터 (Unstructured Data)

- 형태와 구조가 정의되지 않은 데이터. 그림 · 영상 · 문서처럼 구조화되지 않은 형태
- 변칙(anomaly)과 모호함(ambiguity)이 상존
- 예) 뉴스 기사, 이메일, 메신저, SNS 게시물, 음성, 동영상, 로그 → 사용 전 반드시 '정형화' 과정이 필요

전 세계에서 생성되는 데이터의 대부분은 비정형이며, 텍스트는 그중 가장 큰 비중을 차지한다

4.1 비정형 데이터 내의 오류

원시 텍스트(raw text)에는 어떤 오류가 섞여 있는가

1. 구조적 노이즈

- HTML 태그, 마크다운 기호, PDF의 머리말 · 꼬리말 · 쪽번호, 각주 번호
- 본문과 사족(광고, 내비게이션, 저작권 표시)의 구분이 어려움

2. 인코딩 오류

- EUC-KR / CP949 / UTF-8 혼용 → 글자 깨짐(mojibake)
- 한글 자모 분리(NFD) 문제 : '한글' vs 'ㅎ ㅏ ㄴ ㄱ ㅡ ㄹ'

3. 표기 불일치

- 대소문자, 전각 / 반각, 자모, 이모지, 반복 문자('좋아요오오오')

4. 언어적 오류

- 띄어쓰기 오류, 맞춤법 · 오타, 구어체 축약, 신조어와 미등록어

깨진 텍스트의 예 — 모두 같은 문장이지만 기계에게는 전부 다른 문자열이다

- <p>안녕하세요 </p> / ì•ë...•í•ĩ„ìš” / 안 녕 하 세 요 / ㅇ ㅏ ㄴ ㄴ ㅓ ㅇ / 안녕하세요ㅋ
ㅋㅋㅋㅋㅋ

4.1 전처리는 왜 필수적인가

전처리는 모델링에 앞서 수행하는 '데이터 측면'의 성능 개선 작업이다

- 분석을 위해서는 비정형 데이터가 정형화되어야 하며, 이 정형화 과정을 전처리(preprocessing)라고 한다
- 세상에 존재하는 대부분의 가공되지 않은 데이터(raw data)는 비정형 형식을 띠기 때문에, 전처리는 선택이 아니라 필수 단계이다
- 전처리를 올바르게 수행하는 것만으로도 동일한 모델에서 상당한 성능 향상을 기대할 수 있다
 - 어휘 사전(vocabulary)의 크기가 줄어들어 학습이 빨라지고, 미등록어(OOV) 비율이 감소한다
 - 여러 문서 간 표기의 일관성이 확보되어, 이후 단계에서 발생할 오류를 미리 방지할 수 있다

주의

- 전처리는 '많이 할수록 좋은' 작업이 아니다. 감성 분석에서 이모지나 느낌표를 제거하면 오히려 중요한 신호를 잃는다. 목적에 맞는 전처리를 설계해야 한다

핵심

- Garbage In, Garbage Out — 입력 데이터의 품질이 모델 성능의 상한선을 결정한다
- 전처리는 자연어처리를 넘어 데이터를 다루는 모든 작업에서 가장 먼저 수행되는 공통 과정이다

4.1 텍스트 전처리 파이프라인

문서 파일을 '문서'로 바꾸는 표준적인 처리 순서

- 01 텍스트 추출 HTML · PDF · DOCX 에서 본문만 뽑아내기
- 02 정제 (Cleaning) 노이즈 · 특수문자 · 불필요 언어 제거
- 03 정규화 유니코드 · 대소문자 표기 통일
- 04 교정 띄어쓰기 · 맞춤법 · 철자
- 05 토큰화 문장 / 단어 / 서브워드 단위 분할

실무 지침

- 1차적으로 특수문자를 제거하고, 2차적으로 문장과 무관한 특수 커맨드나 코딩 구문을 규칙적으로 제거한다
- PDF처럼 줄바꿈이 무의미하게 삽입된 경우에는 줄바꿈을 무시하고 문장 부호만으로 문장을 구분하도록 처리한다
- 마지막으로 단어를 구분하는 기준(토큰화 기준)을 하나로 통일하여, 일관성 부족으로 생길 수 있는 오류를 방지한다

처리 후 목표 상태

- 입력 문자열에 오직 목표 언어의 문자만 남아 있는 상태. 즉 '문서 파일(file)'이 아니라 '문서(document)'가 된 상태

4.1 텍스트 문서의 변환

문서는 사람이 읽기 쉬운 형식으로 저장되어 있으며, 그 형식은 대개 기계에게 불리하다

HTML 문서

- 문제 : 태그 · 스크립트 · 스타일이 본문과 섞여 있고, 내비게이션 · 광고 · 댓글이 본문처럼 보임
- 해결 : BeautifulSoup 등으로 DOM 파싱 후 텍스트 노드만 추출 / 본문 추출 알고리즘(boilerplate removal) 활용

PDF 문서

- 문제 : 글자의 '좌표'만 저장되어 읽기 순서가 보장되지 않음. 2단 편집 · 표 · 머리말이 뒤섞여 추출되고, 인코딩 해석에 따라 아예 읽을 수 없는 글자로 바뀌기도 함
- 해결 : pdfplumber / PyMuPDF 로 레이아웃 기반 추출, 필요 시 OCR

일반 텍스트 / 로그

- 문제 : 인코딩 정보가 파일에 명시되지 않음. 제어문자 · BOM · 불규칙한 줄바꿈
- 해결 : chardet 등으로 인코딩 추정 후 UTF-8로 통일 / 정규표현식으로 제어문자 제거

공통 원칙

- 사람과 달리 기계는 '어디까지가 본문이고 어디부터가 사족인지' 스스로 구분하지 못한다. 그 판단을 코드로 명시해 주어야 한다

4.1 정제(Cleaning) : 노이즈 제거

정규표현식(Regular Expression)은 전처리에서 가장 자주 쓰이는 도구이다

- 정규표현식은 문자열의 '패턴'을 기술하는 형식 언어로, 특정 패턴을 찾아 제거하거나 치환하는 데 사용한다
- 1차 : HTML 태그, URL, 이메일, 전화번호 등 구조적 노이즈 제거
- 2차 : 목표 언어 이외의 문자(불필요한 타 언어 문자 · 특수문자) 제거
- 3차 : 연속 공백 · 줄바꿈을 하나의 공백으로 축약

제거가 아니라 '치환'이 유리한 경우도 많다

- URL → <URL>, 숫자 → <NUM>, 이메일 → <EMAIL> 로 바꾸면 정보의 존재 자체는 보존된다

빈도 기반 제거

- 극단적으로 드문 단어(오타 · 고유명사)와 지나치게 흔한 단어는 목적에 따라 제외를 검토한다

자주 쓰는 패턴

- `re.sub(r'<[^>]+>', '', t)` # HTML 태그
- `re.sub(r'https?://\S+', '', t)` # URL
- `re.sub(r'^가-힣a-zA-Z ', ' ', t)` # 한글·영문·공백만 남기기
- `re.sub(r'\s+', ' ', t).strip()` # 연속 공백 축약

4.1 정규화(Normalization) : 표기 통일

같은 의미의 표현을 같은 문자열로 만드는 작업

1. 유니코드 정규화 (NFC / NFD / NFKC)

- 한글은 완성형(NFC)과 조합형(NFD)으로 모두 표현 가능하다. macOS에서 만든 파일명이 Windows에서 깨져 보이는 원인이 이것이다
- NFKC는 전각 문자(A B C)를 반각(ABC)으로, 호환 문자를 표준 문자로 통일한다

2. 대소문자 통일 (case folding)

- 'Apple'과 'apple'을 같은 토큰으로. 단, 고유명사 구분이 필요한 과제에서는 적용하지 않는다

3. 반복 문자 축약

- '좋아요오오오오' → '좋아요오오' (반복을 2회로 제한, 3주치의 $(.)\w{1,2}$ 와 동일 기준) 처럼 반복 길이를 제한한다

4. 동의어 · 표기 변형 통일

- '핸드폰 / 휴대폰 / 휴대전화', 'USA / US / United States' 를 사전 기반으로 하나의 대표형으로 매핑한다

4.1 정규화(Normalization) : 표기 통일

같은 의미의 표현을 같은 문자열로 만드는 작업

실습 코드

- `import unicodedata`
- `text = "한글"`
- `nfd = unicodedata.normalize("NFD", text)`
- `nfc = unicodedata.normalize("NFC", nfd)`
- `print(text, len(text))`
- `print(nfd, len(nfd))`
- `print(nfc, len(nfc))`

```
>> 한글 2  
>> 한글 6  
>> 한글 2
```

4.1 토큰화(Tokenization)

토큰(token)이란 분석의 기본 단위이며, 무엇을 토큰으로 삼을지는 설계의 문제이다

영어 토큰화

- 공백 기준 분리가 1차 근사로 잘 동작한다
- 문제 1 축약형 — don't → do + n't / 문제 2 소유격 — John's → John + 's
- 문제 3 하이픈 — state-of-the-art / 문제 4 마침표 — U.S.A. 는 문장 끝이 아님
- 도구 : NLTK word_tokenize, spaCy tokenizer

한국어 토큰화

- 공백(어절) 기준 분리로는 부족하다. 한국어는 교착어로, 어절 하나에 여러 형태소가 결합한다
- '학교에서는' = 학교 + 에서 + 는
- 어절 단위로 토큰화하면 '학교에서', '학교에서는', '학교를' 이 모두 다른 토큰이 되어 어휘가 폭발한다
- → 형태소 단위 토큰화가 표준 (4.2절)

문장 분할(sentence segmentation)도 토큰화의 일부다

- 마침표는 문장 끝일 수도, 약어일 수도, 소수점일 수도 있다

4.1 불용어 제거와 어간 / 표제어 추출

어휘 사전의 크기를 줄이고 단어의 변형을 하나로 묶는 작업

불용어(Stopword) 제거

- 'the, is, at' 또는 '은, 는, 이, 가, 하다' 처럼 빈도는 높지만 변별력이 낮은 단어를 제거한다
- 정보 검색 · 토픽 모델링에서는 효과적이지만, 감성 분석에서 'not'을 제거하면 의미가 뒤집히므로 주의한다

어간 추출 (Stemming)

- 규칙에 따라 접사를 잘라내는 방식. 빠르지만 결과가 실제 단어가 아닐 수 있다 (automates-automation 은 automat 이 아니라 autom 이 된다)
- Porter Stemmer : production / products → product, automatic → automat

표제어 추출 (Lemmatization)

- 사전과 품사 정보를 이용해 사전에 실린 기본형(lemma)을 찾는다. 느리지만 정확하다
- am / are / is → be, better → good (품사 정보 필요)

비교

- Stemming : 규칙 기반 · 빠름 · 사전 불필요 · 결과가 비단어일 수 있음
- Lemmatization : 사전 기반 · 느림 · 품사 필요 · 품사가 맞으면 실제 단어 (품사 미지정 시 was → wa 같은 비단어 가능)
- 한국어는? 어간 / 표제어 추출 대신 형태소 분석기가 그 역할을 수행한다

4.1 띄어쓰기 교정의 중요성

한국어 전처리에서 띄어쓰기 교정은 종의성을 해소하는 매우 중요한 작업이다

- 띄어쓰기가 사용되는 언어에서 띄어쓰기는 단어의 의미 분할과 전달에 결정적인 역할을 한다
- 한국어의 형태소 분석 · 구문 분석은 입력 문장이 어절 단위로 올바르게 띄어쓰기되어 있음을 전제로 연구되어 왔다
- 그러나 실제 웹 텍스트 · SNS · 음성 인식 결과에는 띄어쓰기 오류가 매우 흔하다
- 띄어쓰기가 잘못되면 형태소 분석기가 어절 경계를 잘못 잡아, 이후 모든 분석이 연쇄적으로 실패한다

한국어 띄어쓰기 교정의 세 가지 접근

- ① 규칙 기반 ② 통계 · 확률 기반 ③ 딥러닝 기반

고전적인 예 — 띄어쓰기 하나가 문장 전체의 의미를 바꾼다

- "아버지가방에들어가신다" → 아버지가 방에 들어가신다 / 아버지 가방에 들어가신다
- "무지개같은사람" → 무지 개같은 사람 / 무지개 같은 사람

4.1 띄어쓰기 교정 (1) 규칙 기반

형태소 분석기와 휴리스틱 규칙을 사용하는 분석적 방법

① 원리

- 형태소 분석기를 사용하는 규칙 기반의 분석적 방법
- 규칙은 주로 어휘 지식, 문법 규칙, 오류 유형 등의 휴리스틱으로 구성
- 입력 어절을 사전과 대조하여 분리 가능한 지점을 찾음

② 장점

- 규칙이 적용되는 범위 안에서는 결과가 정확하고 예측 가능
- 왜 그렇게 교정했는지 설명 가능(해석 가능성) / 학습 데이터가 필요 없음

③ 한계

- 형태소 분석기 외에 여러 언어학적 자원을 별도로 구축해야 함
- 여러 단계의 복잡한 휴리스틱 적용 → 분석 과정이 복잡
- 어휘 지식 구축 · 관리 비용이 커서 유지 · 보수가 어려움
- 높은 성능을 요구할수록 시간적 · 인적 비용이 급격히 증가하며, 중의적 구간이나 오타에서 성능이 크게 저하됨

4.1 띄어쓰기 교정 (2) 통계 · 확률 기반

말뭉치에서 자동 추출한 음절 n-gram 정보를 이용한다

- 각 음절 사이의 위치 i 에 대해 $P(\text{space} \mid c(i-2), c(i-1), c(i), c(i+1))$ 을 추정
- $P(w_1..w_n) = \prod P(w_i \mid w_{i-2}, w_{i-1})$ (n-gram 언어 모델)

특징

- 말뭉치로부터 자동 추출된 음절 n-gram 정보를 기반으로 체계적인 계산을 거쳐 오류를 교정하므로 구현이 용이하다
- 어휘 지식을 사람이 직접 구축할 필요가 없고, 미등록어에 대해서도 비교적 견고한 분석이 가능하다

단점

- 학습 말뭉치의 영향을 크게 받으며, 정확도를 개선하려면 대량의 학습 데이터가 필요하다
- 한국어의 경우 신뢰할 만한 n-gram 통계를 얻으려면 띄어쓰기가 올바른 대용량 말뭉치를 확보해야 한다

동작 예시 — 입력 : 아버지가방에들어가셨다

- 아버지가 방에 들어가셨다 0.71 아버지가방에 들어가셨다 0.24 아버지가방에 들어갔
다 0.05
- → 가장 확률이 높은 후보 선택

4.1 띄어쓰기 교정 (3) 딥러닝 기반

띄어쓰기 교정을 음절 단위 이진 분류(sequence labeling) 문제로 정식화한다

- 각 음절 뒤에 공백이 오는가(1) 오지 않는가(0)를 예측하는 문제로 바꾼다
- '아버지가방에' → 아(0) 버(0) 지(0) 가(1) 방(0) 에(1)
- Bi-LSTM + CRF, CNN, Transformer 계열 모델이 널리 사용된다
- 음절 임베딩을 통해 미등록어에도 강건하며, 문맥을 양방향으로 활용할 수 있다
- 사전이나 규칙을 사람이 작성할 필요가 없고, 정답 말뭉치만 있으면 학습이 가능하다

단점

- 대량의 정답 데이터와 학습 자원이 필요하며, 오류의 원인을 설명하기 어렵다(블랙박스)

실무 도구

- Kiwi(space), PyKoSpacing, soyspacing 등의 라이브러리로 손쉽게 적용할 수 있다
- `from kiwipiepy import Kiwi`
- `kiwi = Kiwi(); kiwi.space('아버지가방에들어가신다') # → '아버지가 방에 들어가신다'`

4.1 철자 및 맞춤법 오류의 네 가지 유형

유형	정의	예시 (영어)	예시 (한국어)
삽입(Insertion)	본래 없어야 할 문자를 추가로 입력	the → ther	안녕 → 안녕ㅇ
생략(Deletion)	본래 있어야 할 문자를 빠뜨림	the → th	감사합니다 → 감사합다
대체(Substitution)	넣어야 할 문자 대신 다른 문자를 입력	the → thw	학교 → 학교
전치(Transposition)	인접한 두 문자의 순서가 뒤바뀜	the → hte	그래서 → 그서래
편집 거리	위 연산의 최소 횟수가 편집 거리이며 철자 교정의 기본 척도가 된다. 삽입·생략·대체만 허용하면 Levenshtein distance, 전치까지 허용하면 Damerau-Levenshtein distance 이다		

4.1 편집 거리 (Levenshtein) 계산

- **정의 : 한 문자열을 다른 문자열로 바꾸는 데 필요한 최소 편집 연산 수**
 - 연산 = 삽입 / 삭제 / 대체 (각 비용 1) — 전치까지 허용하면 Damerau-Levenshtein
- **동적 계획법 점화식**
 - $D[i][0] = i, D[0][j] = j$
 - $D[i][j] = \min(D[i-1][j] + 1, D[i][j-1] + 1, D[i-1][j-1] + \text{cost})$
 - $\text{cost} = 0$ (두 문자가 같을 때), 1 (다를 때)
- **예시 : kitten → sitting (정답 3)**
 - ① kitten → sitten (k를 s로 대체)
 - ② sitten → sittin (e를 i로 대체)
 - ③ sittin → sitting (g를 삽입)
- **D 행렬의 마지막 칸 $D[6][7] = 3$ 이 곧 편집 거리이다**
 - 시간·공간 복잡도는 $O(m \cdot n)$ 이며, 두 행만 유지하면 공간은 $O(\min(m, n))$ 로 줄일 수 있다

4.1 맞춤법 교정 : 왜 필요한가

철자 교정기가 수행하는 일은 ① 오류 감지 ② 오류 수정 두 단계이다

- 철자 교정은 정확한 의미 전달과 정보 교환을 위해 반드시 필요하다
- 의미 혼용을 방지하고 정보 전달의 실패를 막기 위해 수행하는 것이 맞춤법 검사이다
- 검색 엔진에서는 질의어의 오타가 곧 검색 실패로 이어지므로, 교정은 검색 품질과 직결된다

① 오류 감지 (Error Detection)

- 사전에 없는 단어(non-word error)를 찾는 것은 비교적 쉽다 — '컴퓨터'
- 그러나 사전에 있지만 문맥상 틀린 단어(real-word error)는 문맥을 봐야 한다
- 예) '나는 학교에 갔다' 와 '나는 학교에 같다' → 언어 모델이 필요

② 오류 수정 (Error Correction)

- 편집 거리가 가까운 후보를 생성하고, 그중 가장 그럴듯한 것을 선택한다
- 교정 방식은 규칙 기반과 통계 · 확률 기반으로 나뉜다

4.1 맞춤법 교정 (1) 규칙 기반

언어 현상의 규칙성을 명시적으로 기술하여 적용하는 방식

처리 절차

- 1. 전처리 — 문장 분리, 정규화 수행
- 2. 품사 태깅 — 각 어절의 형태소 · 품사 정보 부착
- 3. 규칙 탐색 — 오류 패턴 규칙과 매칭
- 4. 교정 결과 제시 — 치환 후보와 근거를 함께 출력

장점

- 규칙에 해당하는 오류는 매우 정확하게 교정
- '왜 틀렸는지' 문법적 근거를 함께 제시할 수 있어 교육용으로 적합 / 학습 데이터 없이 즉시 적용 가능
- 한국어 맞춤법 검사기(부산대 등)가 대표적

문제점 — 모든 규칙 기반 시스템에서 발생하는 문제와 동일하다

- 규칙을 사람이 직접 작성해야 하므로 구축 비용이 크다
- 규칙 수가 늘어날수록 규칙 간 충돌 · 중복 관리가 어려워지고, 신조어 · 구어체 등 규칙 밖의 현상에 대응하지 못한다

4.1 맞춤법 교정 (2) 통계 · 확률 기반

베이저안 추론 모델(Bayesian inference model)에 기반한 잡음 채널 모형

- $\hat{c} = \operatorname{argmax} P(c \mid w) = \operatorname{argmax} P(w \mid c) \cdot P(c)$, $c \in C(w)$
- $P(w|c)$: 오타 모델(channel model) $P(c)$: 언어 모델(prior)

동작 원리

- 관측된 단어 w 가 주어졌을 때, 원래 의도한 단어 c 의 사후 확률이 가장 큰 후보를 선택한다
- $P(w \mid c)$ — 사람이 c 를 쓰려다 w 로 잘못 칠 확률. 편집 거리와 자판 배열의 인접성으로 추정한다
- $P(c)$ — 그 단어가 애초에 등장할 확률. 말뭉치 빈도 또는 n -gram 언어 모델로 추정한다
- 후보 집합 $C(w)$ 는 보통 편집 거리 1~2 이내의 단어들로 제한하여 계산량을 줄인다
- 문맥까지 반영하려면 $P(c)$ 를 n -gram으로 계산하면 되며, 이렇게 하면 real-word error도 교정할 수 있다

계산 예 ($w = \text{'acress'}$)

- $c=\text{across}$ $P(w|c)P(c)=2.8e-9$ $c=\text{actress}$ $2.7e-9$ $c=\text{acres}$ $1.0e-9$ → 'across' 선택. 그러나 실제 정답은 'actress' 였다 - 유니그램 사전확률만으로는 문맥을 반영하지 못하기 때문
- 이후 가장 확률이 높은 후보로 감지한 오타를 대체함으로써 철자 교정을 수행한다
- (Kernighan, Church & Gale, 1990)

4.1 서브워드 토큰화 : 현대적 대안

미등록어(OOV) 문제를 해결하기 위해 단어보다 작은 단위를 사용한다

단어 단위 토큰화의 한계

- 어휘 사전에 없는 단어는 모두 <UNK>
- 한국어 · 터키어 같은 교착어는 활용형이 많아 어휘가 폭발적으로 증가
- 신조어, 오타, 고유명사에 대응 불가. '먹었다 / 먹는다 / 먹겠다'가 전혀 다른 토큰
- → 사전 크기와 OOV 사이의 딜레마

서브워드 토큰화

- 자주 등장하는 문자열 조각을 하나의 토큰으로 학습
- BPE(Byte Pair Encoding) : 가장 빈번한 인접 쌍을 반복적으로 병합
- WordPiece(BERT), SentencePiece(다국어), Unigram LM
- '자연어처리' → '자연' + '##어' + '##처리' → 문자 단위 폴백으로 OOV를 사실상 제거 (원천적 제거는 바이트 수준 BPE)

오늘날의 사전학습 언어모델(BERT, GPT 등)은 모두 서브워드 토큰화를 사용한다

- 다만 형태소 경계와 일치하지 않을 수 있어, 한국어에서는 형태소 분석과 병용하기도 한다

4.1 정리

텍스트 전처리 — 비정형 문자열을 분석 가능한 형태로 만드는 과정

- 텍스트는 비정형 데이터이므로, 사용에 앞서 정형화(전처리)가 반드시 필요하다
- 전처리 파이프라인 : 텍스트 추출 → 정제 → 정규화 → 교정 → 토큰화
- 정제는 정규표현식으로 노이즈를 제거하고, 정규화는 유니코드 · 대소문자 · 표기를 통일한다
- 한국어 띄어쓰기 교정은 규칙 기반 / 통계(n-gram) 기반 / 딥러닝 기반으로 발전해 왔다
- 맞춤법 교정의 오류 유형은 삽입 · 생략 · 대체 · 전치이며, 편집 거리와 잡음 채널 모형이 이론적 토대이다
- 현대 NLP는 OOV 문제를 해결하기 위해 서브워드 토큰화를 표준으로 사용한다

실습 연결

- 실습 노트북 PART 1에서 정규표현식 정제, 유니코드 정규화, NLTK 토큰화, 불용어 제거, Stemming / Lemmatization, Kiwi 띄어쓰기 교정을 직접 수행한다

4.2 어휘 분석

형태소 분석 · 한국어 형태소 분석기 비교 · 품사와 태그셋 · 품사 태깅
접근법 · 활용 분야

4.2 어휘 분석이란

단어의 구조를 식별하고 분석하여 어휘의 의미와 품사를 파악하는 단어 수준의 연구

- 언어의 어휘는 의미의 기본 단위이며, 어휘 분석은 그 기본 단위를 확정하는 작업이다
- 기본적인 어휘적 의미를 판단하고 처리하기 위해서는 두 가지 절차가 필요하다
 - ① 형태소 분석 — 한 문장에서 단어나 구를 식별하고, 최소 의미 단위로 분해한다
 - ② 품사 태깅 — 각 단어에 품사 정보를 부여하여 문법적 역할을 결정한다
- 이 두 과정을 거쳐야 비로소 '어떤 단어가 어떤 자격으로 쓰였는가'를 기계가 알 수 있다
- 어휘 분석의 결과는 구문 분석(4.3)의 입력이 되며, 정보 검색의 색인어 추출에도 그대로 사용된다

왜 필요한가

- "나는 밥을 먹었다" — 표층 문자열만으로는 '나는'이 [나 + 는] 인지 [날다 + 는] 인지 알 수 없다
- → 형태소 분석과 품사 태깅이 이 중의성을 해소한다

4.2 형태소(Morpheme)의 개념

형태소란 더 이상 분해될 수 없는, 최소한의 의미를 갖는 단위이다

① 자립성에 따른 분류

- 자립 형태소 — 혼자 쓰일 수 있는 형태소. 명사, 대명사, 수사, 관형사, 부사, 감탄사 (예 : 하늘, 매우, 아이고)
- 의존 형태소 — 반드시 다른 형태소에 붙어야 하는 형태소. 조사, 어간, 어미, 접사 (예 : -이, -었-, -다, 뜻-)

② 의미에 따른 분류

- 실질(어휘) 형태소 — 구체적인 대상이나 동작을 나타냄 (예 : 하늘, 먹-, 푸르-)
- 형식(문법) 형태소 — 문법적 관계나 기능만을 나타냄 (예 : 이/가, 을/를, -었-, -다)
- 한국어는 형식 형태소가 매우 발달한 교착어이다

③ 분석 예 — "하늘이 매우 푸르렀다"

- 하늘 : 자립 · 실질 / 이 : 의존 · 형식 (주격 조사) / 매우 : 자립 · 실질
- 푸르 : 의존 · 실질 (어간) / 었 : 의존 · 형식 (과거 시제) / 다 : 의존 · 형식 (종결 어미)
- → 3어절이 6개 형태소로 분해된다

4.2 형태소 분석의 절차

자연어의 제약 조건과 문법 규칙에 맞추어 단어의 형성 과정을 분석한다

- STEP 1 형태소 분리 — 단어에서 최소 의미를 포함하는 형태소를 분리
- STEP 2 원형 복원 — 형태론적 변형이 일어난 형태소의 원형을 찾음
- STEP 3 후보 선택 — 결합 조건에 따라 옳은 분석 후보를 선택

STEP 2 (원형 복원)가 한국어에서 특히 어렵다

- 불규칙 활용 때문에 표층형과 기본형이 크게 달라지기 때문이다
- '구웠다' → 굶 + 었 + 다 (ㅂ 불규칙), '들어' → 들 + 어 (ㄷ 불규칙)

STEP 3 (후보 선택)에서 중의성이 발생한다

- 하나의 어절이 여러 방식으로 분석될 수 있기 때문이다
- '감기는' → [감기(명사) + 는] / [감(동사) + 기 + 는] / [감기(동사) + 는]

핵심

- 형태소 분석은 단순한 문자열 분할이 아니라, 사전 · 문법 규칙 · 문맥을 모두 사용하는 탐색 문제이다 → 그래서 품사 태깅과 함께 수행된다

4.2 영어와 한국어의 형태소 분석

언어 유형이 다르면 형태소 분석의 난이도와 방법도 달라진다

영어 (굴절 요소를 가진 분석어)

- 최소 단위의 의미를 갖는 단어를 기본 단위로 한다. 굴절(복수 -s, 과거 -ed)이 남아 있으나 전체적으로는 분석어에 가까워 기능어가 독립 단어로 분리된다
- 'Computers' = 'Computer' + 's' → 두 개의 형태소 / 'unhappiness' = un + happy + ness
- 어절과 단어가 대체로 일치하여 공백 분리만으로도 상당 부분 해결된다
- Stemming과 Lemmatization을 사용하면 형태소 분석을 비교적 쉽게 수행할 수 있다
- 도구 : NLTK(PorterStemmer, WordNetLemmatizer), spaCy(token.lemma_)

한국어 (교착어)

- 하나의 어절에 여러 형태소가 결합하여 어절 ≠ 단어 이다. '학교에서는' = 학교 + 에서 + 는 (3형태소)
- 조사 · 어미의 결합 가능성이 매우 커서 활용형의 수가 폭발적이다
- 불규칙 활용, 축약('되어→돼'), 음운 변동이 빈번하다
- 띄어쓰기 오류와 미등록어(신조어 · 고유명사)의 영향이 크다 → 전용 형태소 분석기가 반드시 필요하다

4.2 한국어 형태소 분석 오픈 라이브러리

분석기	기반	속도	특징
※ KoNLPy	—	—	Hannanum · Kkma · Komoran · Mecab · Okt 5종을 하나의 파 이썬 인터페이스로 제공한다 (Kiwi는 별도 패키지)
Okt (구 Twitter)	Scala / 규칙	보통	구어체·SNS에 강함. 정규화(normalize)와 어구 추출(phrases) 기능 제공
Komoran	Java / 사전	빠름	오타자·띄어쓰기 오류에 비교적 강건. 사용자 사전 추가가 쉬움
Hannanum	Java (KAIST)	보통	오래된 분석기. 어절 단위 분석 특성이 강함
Kkma (꼬꼬마)	Java (서울대)	느림	가장 상세한 태그셋(약 56종). 정확하지만 속도가 느림
Mecab-ko	C++ / CRF	매우 빠름	대용량 처리에 적합. 설치가 다소 번거로움(Windows 주의)
Kiwi	C++ / 통계+신경망	매우 빠름	독립 패키지(kiwipiepy). 띄어쓰기 교정·문장 분리·오타자 대응 내장. pip 설치만으로 사용
선택 기준	—	—	속도(대용량 처리) · 정확도 · 도메인(구어체/문어체) · 사용자 사 전 지원 · 설치 편의성

4.2 형태소 분석기별 결과 비교

같은 문장도 분석기마다 다른 결과를 낸다 — 절대적인 정답은 없다

- 분석기마다 사용하는 사전, 태그셋, 분석 단위가 다르기 때문에 결과가 달라진다
- 복합명사 처리 : '자연어처리'를 하나의 명사로 볼 것인가, '자연어 + 처리'로 나눌 것인가
- 신조어 · 고유명사 : 사전에 없으면 엉뚱하게 분해된다 (미등록어 문제)
- 태그셋 크기 : Kkma는 약 56종, Okt는 약 19종으로 상세도가 다르다
- 따라서 프로젝트 초기에 반드시 자신의 데이터로 여러 분석기를 비교 실험해야 한다

미등록어 대응 전략

- ① 사용자 사전(user dictionary)에 도메인 용어를 등록한다
- ② 통계적 단어 추출(soynlp 등)로 말뭉치에서 신조어를 자동 발굴한다
- ③ 서브워드 토큰화를 병용하여 OOV를 회피한다

"아버지가 방에 들어가신다" 비교

- Okt : 아버지/Noun 가/Josa 방/Noun 에/Josa 들어가신다/Verb
- Kiwi : 아버지/NNG 가/JKS 방/NNG 에/JKB 들어가/VV 시/EP ㄴ다/EF → 어미 분해 수준이 다르다

4.2 품사의 개념

품사란 단어를 기능 · 형태 · 의미에 따라 나눈 갈래이다

① 체언 · 용언

- 체언 (주체 기능) — 명사, 대명사, 수사. 문장에서 주어 · 목적어가 되는 자리
- 용언 (활용 기능) — 동사, 형용사. 어간에 어미가 붙어 활용하며 문장의 서술어가 된다

② 수식언 · 독립언

- 수식언 (수식 기능) — 관형사(체언 수식), 부사(용언 수식). 예) 새 옷, 매우 빠르다
- 독립언 (독립 기능) — 감탄사. 다른 성분과 문법적 관계를 맺지 않는다. 예) 아, 여보세요

③ 관계언

- 관계언 (관계 기능) — 조사. 체언 뒤에 붙어 문법적 관계를 표시한다
- 격조사(이/가, 을/를), 보조사(은/는, 도), 접속조사(와/과)
- 한국어의 특징 : 조사가 문법 관계를 직접 표시하므로 어순이 비교적 자유롭다

태그셋

- 형태소 분석기에서는 문법 범주를 벗어나지 않는 범위에서, 분석에 용이한 수준으로 품사를 재분류한 '태그셋(tagset)'을 설정하여 사용한다

4.2 품사 태그셋 (Tagset)

태그	의미	예	체계
※ 태그셋	분석기가 부여하는 품사 기호의 체계이며, 언어와 도구마다 다르다		
NNG / NNP	일반 명사 / 고유 명사	학교·사과 / 한국·세종대왕	세종 태그셋 (한국어)
VV / VA	동사 / 형용사	먹다·가다 / 예쁘다·크다	세종 태그셋
JKS / JKO	주격 조사 / 목적격 조사	이·가 / 을·를	세종 태그셋
EP / EF	선어말 어미 / 종결 어미	-었-, -시- / -다	세종 태그셋
NN / NNS	명사 단수 / 복수	dog / dogs	Penn Treebank (영어)
VB / VBD / VBZ	동사 원형 / 과거 / 3인칭단수	eat / ate / eats	Penn Treebank
JJ / RB	형용사 / 부사	quick / quickly	Penn Treebank
UPOS 17종	국제 표준 Universal Dependencies. 다국어 비교 연구에 사용	NOUN, VERB, ADJ, ADP, ...	UD

4.2 품사 태깅이란

각 토큰(영어는 단어, 한국어는 형태소)에 정확히 하나의 품사를 부여하는 작업

- 태깅(tagging)이란 같은 단어라도 문맥에 따라 의미가 다를 수 있으므로, 그 자격을 표시해주는 것이다
- 많은 단어가 형태론적으로 중의성을 가지므로, 품사 태깅은 필연적으로 '중의성 해소'를 수반한다
- 영어 : 'book'은 명사(책)일 수도 동사(예약하다)일 수도 있다
- 한국어 : '나는'은 [나+는] 또는 [날+는], '못'은 명사(도구) / 부사(부정)일 수 있다
- 형태론적 중의성을 해결하려면 반드시 좌우에 사용된 단어들의 정보, 즉 문맥(context)을 고려해야 한다
- → 결국 품사 태깅은 '단어 열이 주어졌을 때 가장 그럴듯한 태그 열을 찾는' 순차 레이블링(sequence labeling) 문제이다

중의성 예시

- I book a flight. → book/VB (동사) I read a book. → book/NN (명사)
- 나는 학교에 간다. → 나/NP + 는/JX 새가 하늘을 나는 모습 → 날/VV + 는/ETM

4.2 품사 태깅 (1) 규칙 · 지식 기반

언어 정보를 규칙의 형태로 표현하고 이를 적용하여 태깅을 수행한다

① 규칙의 종류

- 문맥틀 형식의 규칙 — 앞뒤 단어의 태그 패턴으로 조건 기술
- 제약 문법(Constraint Grammar) — 가능한 태그 중 불가능한 것을 제거해 나감
- 긍정 정보 = 문장에서 선호되는 어휘 태그에 대한 언어 지식 / 부정 정보 = 나타날 수 없는 조합

② 구축 방식

- 수동 구축 — 언어 전문가가 규칙을 직접 작성. 정해진 규칙 범위에서는 정확도가 높으나 시간 · 노력이 많이 들어 지식 병목(knowledge bottleneck) 현상 발생
- 자동 / 반자동 구축 — 최소한의 규칙에서 출발해 말뭉치로부터 규칙을 학습 (예 : Brill Tagger). 코퍼스에 의존적이라는 단점

③ 평가

- 장점 — 품사 사이의 관계뿐 아니라 어절 단위에서도 높은 정확도를 보이며, 통계 기반이 다루지 못하는 부분을 규칙으로 교정할 수 있음
- 한계 — 대량의 태그 부착 코퍼스가 구축되면서 통계 기반 방법이 선호되는 추세. 규칙 유지 · 보수 비용이 지속적으로 발생

4.2 품사 태깅 (2) 통계 기반 — HMM

은닉 마르코프 모형(Hidden Markov Model)은 통계적 품사 태깅의 고전이다

- $T^{\wedge} = \operatorname{argmax} P(T \mid W) = \operatorname{argmax} P(W \mid T) \cdot P(T)$
- $\approx \operatorname{argmax} \prod P(w_i \mid t_i) \cdot P(t_i \mid t_{i-1})$
- 방출 확률(emission) 전이 확률(transition)

원리

- 태그가 부착된 대량의 코퍼스가 주어지면, 태깅에 적합한 모델을 선정하고 코퍼스에서 추출한 통계 정보를 이용한다
- 전이 확률 $P(t_i \mid t_{i-1})$ — 관형사 다음에 명사가 올 확률처럼, 태그 사이의 연쇄 확률
- 방출 확률 $P(w_i \mid t_i)$ — 태그 '명사'가 단어 '학교'로 실현될 확률
- 두 확률은 코퍼스에서 단순 빈도 계수로 추정할 수 있어 학습이 매우 빠르다
- 가능한 태그 열의 수는 지수적으로 많지만, 비터비 알고리즘(Viterbi)으로 $O(n \cdot |T|^2)$ 에 최적해를 구한다
- 그 밖의 통계적 방법 : 결정 트리, 최대 엔트로피 모형, 베이지언 추론, 신경망, Labeling 기법 등

장단점

- 장점 — 코퍼스만 있으면 통계 정보 추출이 용이하고 자동화 가능. 정확도 95% 이상 달성
- 단점 — 대량의 태그 부착 코퍼스가 반드시 필요, 미등록어에 취약, 직전 태그만 보는 제한된 문맥

4.2 품사 태깅 (3) 딥러닝 기반

언어 처리에서 딥러닝이 갖는 효과는 무엇인가

- ① 데이터로부터 특징(feature)을 자동으로 학습한다 — 사람이 자질을 설계할 필요가 없다
- ② 폭넓은 문맥 정보를 다룰 수 있다 — RNN / Transformer는 문장 전체를 참조한다
- ③ 모델에 적합한 출력 형태를 다루기가 간단하다 — 분류, 계열, 생성으로 자유롭게 구성
- ④ 사진 · 음성 등 특성이 다른 모델과 상호작용할 수 있어 멀티모달(Multi-modal) 구축이 쉽다

대표적인 모델 구조

- Bi-LSTM + CRF : 양방향 문맥 + 태그 간 전이 제약을 함께 학습하는 표준 구조
- Seq2Seq : 인코더로 입력을 표현하고 디코더로 형태소 · 태그를 차례로 출력. 한국어는 ㉠ 형태소 분할 · 변환 ㉡ 태그 부여의 두 단계가 필요하며, Seq2Seq는 이를 한 번에 처리할 수 있다
- BERT 등 사전학습 모델 : 대규모 말뭉치로 사전학습 후 태깅 층만 미세조정 → 현재 최고 성능

성능 추이 (한국어)

- 규칙 기반 약 90% 내외(사전 의존) → HMM / CRF 약 95~96% → Bi-LSTM-CRF 약 97% → BERT 기반 약 98% 이상
- 다만 미등록어 · 구어체에서는 여전히 오류가 발생한다

4.2 형태소 분석 및 품사 태깅의 활용 분야

어휘 분석은 그 자체가 목적이라기보다 다른 과제의 기반 기술이다

- 구문 분석의 전처리 — 형태소 분석과 품사 태깅은 구문 분석 단계의 전처리 과정으로 간주되기도 한다
- 정보 검색 — 높은 정확률(precision)과 재현율(recall)을 갖는 색인어 · 검색어 추출에 사용된다. 조사와 어미를 제거하고 실질 형태소만 색인하면 검색 품질이 크게 향상된다
- 기계 번역 — 원문의 문법 정보를 파악하여 대역어를 선택하는 데 사용된다
- 질의응답 시스템 및 언어 정보 획득 — 질문의 초점(focus)과 정답 유형을 파악한다
- 철자 검사 및 교정, 사전 구축 — 4.1절의 교정 기법이 품사 정보를 필요로 한다
- 텍스트 마이닝 — 명사 · 동사만 추출하여 키워드 분석, 워드클라우드, 토픽 모델링을 수행한다
- 국어 연구 · 교육 분야 — 말뭉치 언어학, 사전 편찬, 국어 교육 자료 분석에 활용된다

정리

- 형태소 분석 · 품사 태깅은 자연어처리 제반 분야에서 가장 필수적인 기반 시스템으로 활용되고 있다

4.2 정리

어휘 분석 — 문자열을 '의미를 가진 단위'로 확정하는 단계

- 어휘 분석은 형태소 분석과 품사 태깅의 두 절차로 구성된다
- 형태소는 최소 의미 단위이며, 자립 / 의존 · 실질 / 형식으로 분류된다
- 한국어는 교착어이므로 어절 ≠ 단어이며, 전용 형태소 분석기(Okt, Komoran, Kkma, Mecab, Kiwi)가 필요하다
- 품사 태깅은 형태론적 중의성 해소 문제이며, 문맥 정보가 필수적이다
- 접근법은 규칙 기반 → 통계 기반(HMM + Viterbi) → 딥러닝 기반(Bi-LSTM-CRF, BERT)으로 발전해 왔다

실습 연결

- 실습 노트북 PART 2에서 Kiwi와 Okt로 형태소 분석 결과를 비교하고, NLTK로 영어 품사 태깅을 수행한 뒤 품사별 빈도를 분석한다

4.3 구문 분석

구문 분석 개요 · 구구조 문법과 의존 문법 · 구문 중의성 · 구구조 구문 분석 · 의존 구문 분석 · 접근 방법의 장단점

4.3 구문 분석 개요

자연어 문장에서 구성 요소들의 문법적 구조를 분석하는 기술

- 구문 분석은 문법적 구조 정보를 자동으로 추출함으로써, 기계 번역 · 정보 검색 · 전문가 시스템 등에서 문장 의미의 분석을 돕는 세부 기술로 활용된다
- 어휘 분석이 '단어'를 확정했다면, 구문 분석은 그 단어들이 '어떻게 연결되는가'를 밝힌다
- 구문 문법(syntactic grammar)이란 문법적 구성 요소로부터 문장을 생성하고, 반대로 문장을 구성 요소로 분석하는 문법이다
- 구문 분석의 목표는 자연어 문장의 문법적 구조를 구문 문법에 따라 자동으로 분석하는 것이다
- 따라서 구문 문법을 정의하는 것은 곧 '추출하고자 하는 문법 구조 정보'를 정의하는 것과 같다
- 구문 분석 과제에서도 규칙 기반 · 통계 기반 · 딥러닝 기반 방법들이 차례로 제안되어 왔다

무엇을 얻는가 — "나는 빨간 모자를 쓴 소녀를 보았다"

- '빨간'은 '모자'를 수식 / '쓴'은 '소녀'를 수식 / '보았다'의 목적어는 '소녀'
- 이 관계 정보 없이는 번역도 요약도 정확할 수 없다

4.3 두 갈래의 구문 문법

구문 분석에서 대표적으로 활용되는 두 가지 문법 체계

구구조 문법 (Phrase Structure Grammar)

- 구성소(constituent) 관계에 기반하여 문장 구조를 분석한다
- 단어들이 모여 절(구)을 구성하며, 이러한 절과 단어들의 계층적 관계에 따라 문장이 이루어진다고 본다
- 문장 전체를 트리로 분석할 때 단어는 잎(leaf) 노드가 되고, 내부 노드는 구(NP, VP)가 된다
- 구성소란 한 개의 단위처럼 기능하는 일련의 단어를 말한다 → 어순이 비교적 고정적인 언어(영어)에 적합

의존 문법 (Dependency Grammar)

- 의존(dependency) 관계에 기반하여 문장 구조를 분석한다
- 문장을 구성하는 단어들 간의 계층적인 의존 관계에 따라 문장이 이루어진다고 본다
- 트리에서 단어들은 각각 노드로 표현되며, 에지는 단어 간의 의존 관계를 나타낸다
- 절의 중심이 되는 단어를 지배소(head), 그에 의존하는 단어를 의존소(dependent)라 한다 → 어순이 유연한 언어(한국어)에 적합

구구조 문법은 '무엇이 무엇과 묶이는가', 의존 문법은 '무엇이 무엇에 걸리는가'를 기술한다

4.3 구구조 트리 (Constituency Tree)

"철수가 사과를 먹었다" 의 구구조 분석

- (S (NP 철수가) (VP (NP 사과를) (V 먹었다))) ← 괄호 표기
- S
 - └─ NP → 철수가
 - └─ VP
 - └─ NP → 사과를
 - └─ V → 먹었다

문맥 자유 문법(CFG)

- $S \rightarrow NP VP$ $VP \rightarrow NP V$ $NP \rightarrow \text{철수가} \mid \text{사과를}$ $V \rightarrow \text{먹었다}$
- 비단말 기호 : S, NP, VP, V / 단말 기호 : 실제 단어 / 시작 기호 : S
- 규칙을 반복 적용하여 문장을 생성(generation)하거나 역으로 분석(parsing)한다

4.3 의존 트리 (Dependency Tree)

"철수가 빨간 사과를 먹었다" 의 의존 분석

- 먹었다 (ROOT)
- └─ 철수가 NP_SBJ (주어)
- └─ 사과를 NP_OBJ (목적어)
- └─ 빨간 VP_MOD (수식)

읽는 법 — 각 에지는 지배소(head) → 의존소(dependent)

- 먹었다 → 철수가 (주어, NP_SBJ)
- 먹었다 → 사과를 (목적어, NP_OBJ)
- 사과를 → 빨간 (수식, VP_MOD)

특징

- 노드 수 = 단어 수 (내부 노드 없음)
- 문장의 최종 지배소는 대개 서술어
- 한국어에서는 의존 구문 분석에서는 지배소가 의존소보다 뒤에 온다고 가정한다 (구어의 도치문은 예외) (head-final)

4.3 구문 중의성 (Syntactic Ambiguity)

자연어 문장의 구문 구조가 여러 방식으로 분석될 수 있는 특징

- 하나의 문장에서 가능한 구문 구조가 여러 개일 때 구문 중의성이 발생한다
- "예쁜 친구의 동생" — 예쁜 것이 친구인가, 동생인가? (수식 범위 중의성)
- "I saw the man with a telescope" — 망원경을 가진 사람인가, 망원경으로 본 것인가? (전치사 구 부착 중의성)

중의성의 대표 유형

- ① 부착 중의성(attachment ambiguity) — 수식어가 어디에 걸리는가
- ② 등위 접속 중의성(coordination) — '늙은 남자와 여자'
- ③ 명사구 경계 중의성 — '한국 대학 축구 연맹'의 묶음 방식
- 구문 중의성의 해소를 위해서는 의미 · 문맥 등의 추가적인 정보가 필요하다

왜 어려운가

- 문장이 길어질수록 가능한 구문 트리의 수는 카탈란 수(Catalan number)로 폭발적으로 증가한다
- 단어 n 개의 이진 트리 수 = $C(n-1)$: $n=4 \rightarrow 5$ 개, $n=6 \rightarrow 42$ 개, $n=9 \rightarrow 1,430$ 개
- $\rightarrow$ 모든 후보를 나열할 수 없으므로 확률 · 학습 기반 선택이 필요하다

4.3 구구조 구문 분석 (1) 규칙 기반

인간의 언어학적 지식을 컴퓨터가 이해할 수 있는 문법 규칙으로 미리 정의한 뒤 적용한다

① 문맥 자유 문법 (CFG)

- 구구조 문법은 자연어 문장을 하위 구성소로 나눔으로써 문장 구조를 나타내는 문법이다
- $G = (N, \Sigma, R, S)$ 로 정의된다
 - N : 비단말 기호 집합 ($S, NP, VP \dots$) / Σ : 단말 기호 집합 (단어)
 - R : 생성 규칙 ($A \rightarrow \beta$) / S : 시작 기호

② 파싱 알고리즘

- 하향식 (Top-down) — S 에서 출발해 규칙을 전개하며 문장과 맞춰본다
- 상향식 (Bottom-up) — 단어에서 출발해 규칙을 거꾸로 적용한다
- 차트 파싱 (CYK, Earley) — 부분 결과를 표에 저장해 중복 계산을 제거 (동적 계획법)

③ 장점과 한계

- 장점 — 미리 정의된 규칙이 적용되는 문장에 대해서는 정확한 분석이 가능하고, 분석 결과의 근거가 명확함
- 한계 — 실제 문장의 다양성을 규칙으로 모두 포괄할 수 없고, 중의성이 발생했을 때 어느 트리가 옳은지 규칙만으로는 선택할 수 없음

4.3 구구조 구문 분석 (2) 통계 기반

확률적 구구조 문법을 통계적으로 계산하여 구문 분석을 수행하는 접근 방법

- $P(T) = \prod P(A \rightarrow \beta \mid A)$ 각 규칙에 조건부 확률을 부여
- $T^{\wedge} = \operatorname{argmax} P(T \mid S) \rightarrow$ 확률이 가장 높은 파스 트리를 선택

원리

- 문법 규칙은 확률적 구구조 문법(PCFG)으로 표현되며, 각 규칙에 조건부 확률이 정의된다
- 확률 추정 ① : 인간이 직접 태깅한 구구조 구문 분석 코퍼스(트리뱅크)로부터 각 규칙의 조건부 확률을 계산한다
- 확률 추정 ② : 태깅되지 않은 문장들에 구문 분석을 반복 수행하며 확률을 갱신한다(EM, inside-outside 알고리즘)
- 덕분에 구문 중의성을 갖는 문장에 대해서도 여러 분석 결과 각각의 확률을 계산한 뒤, 가장 타당할 것으로 예상되는 결과를 선택할 수 있다
- 확률 CYK 알고리즘으로 최대 확률 트리를 효율적으로 찾는다
- 한계 : 규칙의 확률이 문맥과 무관하게 고정되어, 어휘 정보(lexical head)를 반영하지 못한다 $\rightarrow$ Lexicalized PCFG로 보완

대표 자원과 성능

- Penn Treebank(영어 구구조 트리뱅크, 약 100만 어절), 세종 구문 분석 말뭉치(한국어)
- 성능 지표 F1 (labeled bracketing) : 영어 PCFG $\approx 73\%$, 현대 신경망 파서 $\approx 95\%$

4.3 구구조 구문 분석 (3) 딥러닝 기반

인간이 구축한 구구조 데이터셋으로부터 딥러닝 모델을 학습하여 구문 분석을 수행한다

- 딥러닝 기반 방법들은 규칙 기반 · 통계 기반 방법의 틀을 유지하면서, 확률 추정 부분을 신경망으로 대체한다
- 대표적인 방법으로 전이 기반(transition-based) 방식이 있다
- 자연어 문장을 한 단어씩 읽으며 현재 단계에서 수행할 액션(SHIFT, REDUCE 등)을 선택하는 방식으로 문장 전체의 구구조를 구성한다

장점

- 입력 문장에 포함된 단어 수에 선형적인 수의 전이 액션으로 구문 분석이 가능하다 ($O(n)$). 탐색 공간이 좁아 속도가 매우 빠르다

한계

- 각 전이 액션을 선택할 때 문장 전체의 문법적 구조를 고려하기 어렵다 (지역적 결정)
- 문장 내에서 거리가 먼 단어들 간의 관계를 분석하기 어렵고, 한 번의 잘못된 액션이 이후 모든 결정에 영향을 주는 오류 전파(error propagation)에 취약하다

보완 기법

- 빔 서치(beam search) — 상위 k 개 후보를 동시에 유지하여 지역 최적해를 회피
- 동적 오라클 — 학습 시 잘못된 상태에서도 최선의 액션을 학습
- 사전학습 인코더 — BERT로 문장 전체를 표현한 뒤 액션을 선택하여 장거리 의존을 보완

4.3 의존 구문 분석

단어 간의 의존 관계를 분석함으로써 문장 전체의 문법적 구조를 밝히는 기술

- 대부분의 의존 관계 표현은 각 단어를 노드로 하고, 단어 간의 의존 관계를 에지로 나타내는 트리 구조를 따른다
- 의존 분석은 단어 간의 의존 관계와 그 유형을 분석하여, 문장의 문법적 구조를 표현하는 의존 트리를 구축하는 과제로 볼 수 있다
- 의존 문법과 의존 구문 분석은 문장 구조가 비교적 유연한 언어에 적합하다

의존 트리가 만족해야 하는 조건

- ① 단일 지배소 — 루트를 제외한 모든 노드는 정확히 하나의 지배소를 가진다
- ② 비순환 — 순환이 존재하지 않는다 ③ 연결성 — 루트에서 모든 노드에 도달할 수 있다
- 사영성(projectivity) — 에지가 서로 교차하지 않는 성질. 한국어 · 영어는 대체로 사영적이지만, 어순 도치가 일어나면 비사영 구조가 나타난다

한국어의 특성

- ① 지배소 후위(head-final) — 지배소는 의존소의 오른쪽에 온다고 가정한다 (도치문은 비사영 구조로 처리)
- ② 조사가 관계를 표시 — 이/가 → 주어, 을/를 → 목적어. 덕분에 어순이 바뀌어도 관계를 알 수 있다
- ③ 어절 단위 분석 — 한국어는 형태소가 아닌 어절을 노드로 삼는 경우가 많다

4.3 의존 구문 분석의 두 가지 접근

딥러닝 기반 의존 구문 분석은 크게 전이 기반과 그래프 기반으로 구분된다

전이 기반 파싱 (Transition-based)

- 문장의 단어를 하나씩 의존 트리에 포함시키며 분석한다
- 상태 = (스택, 버퍼, 지금까지의 에지) / 액션 : SHIFT / LEFT-ARC / RIGHT-ARC
- 각 시점에서 분류기가 다음 액션을 예측한다
- 장점 — $O(n)$ 선형 시간, 매우 빠름 단점 — 지역적 결정 → 오류 전파, 장거리 의존에 약함

그래프 기반 파싱 (Graph-based)

- 가능한 모든 단어 쌍의 의존 관계에 점수를 매긴 뒤 최적 트리를 찾는다
- 완전 그래프에서 최대 신장 트리(MST)를 구한다 (Chu-Liu / Edmonds)
- Biaffine attention 파서가 대표적 (Dozat & Manning, 2017)
- 장점 — 문장 전체를 동시에 고려 → 장거리 의존에 강함, 전역 최적 단점 — $O(n^2) \sim O(n^3)$ 계산 비용

규칙 기반 의존 분석에서도 의존 문법의 형태로 규칙을 저장한 뒤 적용하며, 통계 기반에서는 문법 규칙의 조건부 확률을 계산하여 적용한다

4.3 구문 분석 접근 방법의 장단점

접근 방법	장점	단점
※ 세 접근법	대체 관계가 아니라, 자원과 목적에 따른 선택의 문제이다	
규칙 기반	문장 구조를 컴퓨터가 이해할 수 있는 문법 규칙으로 명시하므로, 규칙이 적용되는 문장에 대해서는 정확한 분석이 가능하다. 결과의 근거를 설명할 수 있다.	규칙을 사람이 구축해야 하므로 비용이 크고, 규칙 밖의 문장이나 중의성 있는 문장을 처리하지 못한다.
통계 기반	문법 규칙의 조건부 확률을 코퍼스로부터 자동 계산한다. 중의성 있는 문장에도 각 분석 결과의 확률을 계산해 가장 타당한 것을 선택할 수 있다.	대량의 트리뱅크가 필요하며, 어휘 정보와 장거리 문맥을 충분히 반영하지 못한다.
딥러닝 기반	문장에 포함된 어휘·문맥 정보를 자동으로 학습하여 가장 적합한 구조를 예측한다. 사전학습 모델과 결합하면 최고 성능을 낸다.	대규모 학습 데이터와 연산 자원이 필요하며, 분석 실패의 원인을 해석하기 어렵다.

4.3 정리

구문 분석 — 단어들의 문법적 관계를 구조로 표현하는 단계

- 구문 분석은 문장의 문법적 구조를 자동으로 추출하여 의미 분석을 돕는 기술이다
- 구구조 문법은 구성소의 계층 관계를, 의존 문법은 단어 간 지배-의존 관계를 기술한다
- 구문 중의성은 자연어의 본질적 특성이며, 해소를 위해 확률 · 의미 · 문맥 정보가 필요하다
- 구구조 분석 : CFG(규칙) → PCFG(통계) → 전이 기반 신경망(딥러닝)
- 의존 분석 : 전이 기반(빠름, 지역적) vs 그래프 기반(정확, 전역적)
- 한국어는 지배소 후위 · 자유 어순 · 조사 표지의 특성 때문에 의존 구문 분석이 주로 사용된다

실습 연결

- 실습 노트북 PART 3에서 NLTK의 CFG 차트 파서로 구구조 트리를 직접 생성하고, spaCy로 영어 문장의 의존 구문 분석 결과를 시각화한다

4.4 의미 분석

단어와 단어 의미 중의성 · 중의성 해소(WSD) 기법 · 의미역 분석 · 의미 표현

4.4 의미 분석 개요

문법적으로 옳은 문장이 항상 하나의 의미만 갖는 것은 아니다

- 의미 분석은 구문 분석의 결과 위에서 '이 문장이 실제로 무엇을 뜻하는가'를 밝히는 단계이다
- 구문 분석까지 마쳐도, 단어의 의미가 확정되지 않으면 문장의 의미는 여전히 모호하다

의미 분석의 주요 과제

- ① 단어 의미 중의성 해소(WSD) — 한 단어의 여러 의미 중 문맥에 맞는 것을 고른다
- ② 의미역 결정(SRL) — 누가, 무엇을, 누구에게, 어디서 했는지를 밝힌다
- ③ 의미 표현 — 의미를 기계가 계산할 수 있는 형태(벡터, 논리식, 그래프)로 나타낸다
- ④ 개체명 인식 · 상호참조 해결 — 무엇을 가리키는 표현인지 연결한다

문법 ≠ 의미

- "Colorless green ideas sleep furiously" (Chomsky, 1957)
- 문법적으로 완벽하지만 의미적으로는 성립하지 않는 문장 → 구문 분석만으로는 의미를 보장할 수 없다

4.4 중의성의 종류

사람이 쓰는 언어 표현에는 둘 이상의 의미를 가지는 표현들이 있는데, 이를 중의성이라 한다

① 어휘적 중의성

- 하나의 단어가 문맥이나 상황에 따라 다른 의미로 해석되는 것
- 다의어(polysemy) — 의미들이 서로 관련됨. '손' : 신체 부위 / 일손 / 도움
- 동음이의어(homonymy) — 형태만 같고 의미는 무관. '배' : 신체 / 과일 / 선박, 'bank' : 은행 / 강둑

② 구조적 중의성

- 문장의 구조가 여러 방식으로 분석되어 발생하는 중의성
- "예쁜 친구의 동생" → [예쁜 친구]의 동생 / 예쁜 [친구의 동생]
- "나는 울면서 떠나는 그녀를 배웅했다" → 우는 사람이 '나'인가 '그녀'인가 (부사절 부착 중의성)
- 4.3절의 구문 중의성과 같은 현상이며, 해소에는 의미 정보가 필요하다

③ 화용적 중의성

- 발화 상황과 화자의 의도에 따라 달라지는 중의성
- "창문 좀 열어 줄 수 있어?" → 능력에 대한 질문(문자적) / 요청(의도)
- 지시 표현 : "그것 좀 줘"의 '그것' → 상호참조 해결(coreference resolution)이 필요하다

중의성 있는 단어를 적절한 문맥 정보와 함께 표현하지 않으면 의사 전달의 대상에게 혼동을 준다
. 정확한 분석을 위해서는 중의적 표현의 의미를 결정하는 외부 정보가 필요하다

4.4 단어 의미 중의성 해소 (WSD)

Word Sense Disambiguation — 문장 내 중의적 어휘를 사전에 정의된 의미와 매칭하는 문제

- 단어 중의성 해소란 문장에 표현된 단어들을 기계가 정확하게 이해하도록, 어휘적 중의성을 해결하는 과제이다
- 사전에 정의된 의미 목록(sense inventory)을 전제로 하며, 한국어는 표준국어대사전 · KorLex, 영어는 WordNet을 주로 사용한다

해소 기법은 크게 세 가지로 나뉜다

- ① 지식 기반 방법 — 사전 · 시소러스 등 어휘 지식을 활용
- ② 지도학습 방법 — 의미가 레이블링된 데이터로 분류 모델을 학습
- ③ 심층학습 방법 — 지식 정보와 레이블 데이터를 함께 활용

WSD가 필요한 대표 응용

- 기계 번역('배'를 ship으로 번역할지 pear로 번역할지), 정보 검색, 질의응답, 감성 분석

WordNet — 영어 어휘 의미망

- 단어를 동의어 집합(synset) 단위로 묶고, 상하위어 · 부분어 관계로 연결한 지식베이스
- bass.n.01 : 낮은 음역 bass.n.08 : 농어(물고기)
- 한국어 : KorLex, 우리말샘, 세종 의미 부착 말뭉치

4.4 WSD 기법 (1) 지식 기반 · 지도학습

레이블 데이터의 유무에 따라 접근 방식이 달라진다

① 지식 기반 방법

- 문장에 등장한 단어들을 사전에 정의된 어휘 지식을 활용하여 예측하는 방법
- Lesk 알고리즘 — 동의어의 각 의미에 대한 사전 정의문(gloss)과 주변 문맥 단어의 중복 개수를 세어, 겹침이 가장 큰 의미를 선택
- 의미망 기반 — WordNet의 그래프 거리로 의미 간 유사도를 계산
- 장점 : 레이블 데이터가 필요 없음 / 한계 : 사전 정의문이 짧아 겹침 정보가 부족

② 지도학습 기반 방법

- 문장 내 단어의 의미가 레이블링된 데이터를 이용해 기계학습 모델을 학습하고 의미를 예측하는 방법
- 주요 자질(feature) : 주변 단어(window) 및 그 품사 / 연어(collocation) 정보 / 구문 의존 관계
- 모델 : Naive Bayes, SVM, 결정 트리 / 한계 : 단어마다 별도의 학습 데이터 필요

③ 준지도 · 비지도 — 레이블 부족 문제를 완화하는 절충 방법

- Yarowsky 알고리즘 — '하나의 담화에는 하나의 의미', '하나의 연어에는 하나의 의미'라는 가정으로 소량의 시드에서 레이블을 확장
- 의미 유도(sense induction) — 문맥 벡터를 군집화하여 의미를 자동으로 발견. 적은 레이블로 대규모 데이터 활용 가능

4.4 WSD 기법 (2) 심층학습 기반

문맥 임베딩(contextual embedding)의 등장으로 WSD를 근본적으로 바꾸었다

- Word2Vec · GloVe 같은 정적 임베딩은 한 단어에 하나의 벡터만 부여하므로, 다의어를 구분할 수 없다는 근본적 한계가 있었다
- '배'라는 단어는 문맥과 무관하게 항상 같은 벡터가 된다 → 중의성 해소 불가
- ELMo · BERT 등 문맥 임베딩은 같은 단어라도 문맥에 따라 다른 벡터를 생성한다
- '배가 아프다'의 '배'와 '배를 탔다'의 '배'가 서로 다른 벡터로 표현된다
- 따라서 각 의미(synset)의 대표 벡터를 만들어 두고, 입력 문맥 벡터와 가장 가까운 의미를 고르는 방식으로 WSD를 수행할 수 있다

지식 정보와 레이블 데이터를 함께 활용하는 방식

- GlossBERT — 사전 정의문과 문맥을 함께 입력하여 '이 정의가 맞는가'를 이진 분류로 판별
- EWISE / ARES — WordNet 그래프 정보를 임베딩 공간에 함께 반영
- 현재 영어 WSD 벤치마크에서 사람 수준(약 80% 내외의 합의 정확도)에 근접한 성능을 보인다

분포 가설

- "You shall know a word by the company it keeps." — J.R. Firth (1957)
- 비슷한 문맥에 나타나는 단어는 비슷한 의미를 가진다는 가설. Word2Vec부터 BERT까지 모든 의미 표현 기법의 이론적 토대이다

4.4 의미역(Semantic Role) 분석

문장의 의미가 올바르게 해석되려면 각 단어들의 관계가 정확하게 형성되어야 한다

- 의미역 분석이란 의미를 해석하기 위해, 서술어가 취하는 대상들의 의미 관계를 파악하고 그 역할을 분류하는 것이다
- 의미역은 문법 구조에 반영되는 의미의 측면을 파악하기 위해 고안되었으며, 상황에 대한 고정된 해석을 전제로 한다
- 따라서 구조적으로 중의성을 가지는 문장의 의미도 정확하게 이해할 수 있다

핵심 아이디어 : 문법적 격(格)과 의미적 역할은 일치하지 않는다

- "철수가 문을 열었다" — 철수 = 주어 = 행동주(Agent)
- "열쇠가 문을 열었다" — 열쇠 = 주어 = 도구(Instrument)
- "문이 열렸다" — 문 = 주어 = 피동주(Theme)
- → 표층 구조(주어 / 목적어)가 같아도 의미역은 다르다. 의미역을 알아야 '누가 실제 행위자인지'를 알 수 있다

SRL이 답하는 질문 : Who did What to Whom, Where, When, How, Why

- "어제 철수가 도서관에서 영희에게 책을 주었다"
- 술어 : 주다 / 행동주 : 철수 / 피동주 : 책 / 수령자(도달점) : 영희 / 장소 : 도서관 / 시간 : 어제

4.4 의미역의 종류

의미역은 서술어에 대한 필수성 여부에 따라 두 가지로 나뉜다

필수 / 수의 여부는 의미역의 종류가 아니라 서술어가 요구하는가로 결정된다 - 예)
주다는 행동주·피동주·도달점이 필수, 놓다는 장소가 필수이며, 도구·수혜자는 대개
수의적이다

- 행동주(Agent) — 행위를 하는 주체 피동주(Theme / Patient) — 행위의 대상
- 경험자(Experiencer) — 심리 · 지각의 주체 수혜자(Beneficiary) — 이익을 받는 대상
- 도구(Instrument) — 행위에 사용된 수단 출처(Source) — 이동의 출발점
- 도달점(Goal) — 이동의 도착점

수의적 의미역 (Adjuncts) — 문장에서 서술어의 의미를 보충하는 부가 성분

- 장소 / 위치(Location) 이유(Cause) — 왜 목적(Purpose) — 무엇을 위해
- 경로(Path) — 어디를 거쳐 시간(Time) — 언제 방법(Manner) — 어떻게
- 없어도 문장이 성립하지만, 있으면 상황이 구체화된다

의미역 기준(theta-criterion)

- 통사적 분석에서 하나의 명사구는 오직 하나의 의미역만을 가진다

4.4 의미역 결정(SRL)의 방법론

SRL은 ① 술어 식별 ② 논항 식별 ③ 의미역 분류 의 단계로 수행된다

지도학습 기반 방법

- CRFs, SVM 등의 분류기를 주로 사용한다
- 분류기의 입력으로 다양한 자질(feature)을 사용하며, 대표적인 자질로는 형태소 정보와 구문 정보가 사용된다
- 그 밖의 자질 : 술어의 어휘, 논항의 핵심어, 술어-논항 간 구문 경로(path), 격조사, 위치 관계
- 구문 분석 결과가 SRL의 입력이 되므로, 4.3절의 품질이 4.4절의 성능을 좌우한다

딥러닝 기반 방법

- Bi-LSTM 기반 계열 레이블링으로 구문 분석 없이도 높은 성능 달성 (He et al., 2017)
- BERT 기반 : 문장과 술어를 함께 입력하여 각 토큰의 의미역을 BIO 태그로 예측

주요 자원

- PropBank(술어 중심, ARG0~ARG5), FrameNet(프레임 의미론), 한국어 PropBank · 세종 의미역 말뭉치

BIO 태깅 예

- 철수가 B-ARG0 영희에게 B-ARG2 책을 B-ARG1 주었다 B-V
- ARG0 = 행동주, ARG1 = 피동주, ARG2 = 수령자(도달점), V = 술어
- '주다'의 '영희에게'는 소유권 이동의 도착점이므로 수혜자(Beneficiary)가 아니라 도달점(Goal) · 수령자(Recipient)로 분류한다

4.4 의미 표현

분석한 의미를 기계가 계산할 수 있는 형태로 나타내는 방법

① 기호적 표현

- 논리식 기반 — 1차 술어 논리로 문장을 표현. $\exists x. 사과(x) \wedge 먹다(철수, x)$
- 추론이 가능하고 정확하지만, 자연어 전체를 포괄하기 어렵다
- 의미망 · 온톨로지 — 개념 간 관계를 그래프로 표현 (WordNet, KorLex, 지식 그래프)
- AMR (Abstract Meaning Representation) — 문장의 의미를 방향 그래프로 표현

② 분산 표현 (임베딩)

- 분포 가설에 기반해 단어를 실수 벡터로 표현
- 정적 임베딩 : Word2Vec(CBOW / Skip-gram), GloVe, FastText. $king - man + woman \approx queen$
- 한 단어 = 한 벡터 $\rightarrow$ 다의어 구분 불가
- 문맥 임베딩 : ELMo, BERT, GPT. 같은 단어라도 문맥마다 다른 벡터 $\rightarrow$ 현재 의미 표현의 표준

③ 활용

- 문장 · 문서 임베딩 — Sentence-BERT로 문장 전체를 벡터화하여 의미 유사도를 계산
- 의미 검색 — 키워드가 아닌 의미로 검색 (dense retrieval) / RAG — 질문과 의미적으로 유사한 문서를 찾아 생성 모델에 제공
- $\rightarrow$ 의미 분석은 오늘날 검색 · 대화 시스템의 핵심 기술이 되었다

4.4 정리

의미 분석 — 구조 위에서 '무엇을 뜻하는가'를 확정하는 단계

- 중의성은 어휘적 · 구조적 · 화용적 중의성으로 나뉘며, 해소에는 문맥 정보가 필수적이다
- 단어 의미 중의성 해소(WSD)는 지식 기반 · 지도학습 · 심층학습 방법으로 접근한다
- 문맥 임베딩(BERT)의 등장으로 다의어 표현의 근본적 한계가 해소되었다
- 의미역 분석은 '누가 무엇을 누구에게' 했는지를 밝히며, 필수적 / 수의적 의미역으로 구분된다
- 의미 표현은 논리식 · 의미망 같은 기호적 표현과, 임베딩 같은 분산 표현으로 나뉜다

실습 연결

- 실습 노트북 PART 4에서 WordNet으로 의미 목록을 조회하고, Lesk 알고리즘으로 WSD를 수행하며, 문맥 임베딩으로 다의어의 의미 차이를 확인한다

WRAP-UP 요약

CHAPTER 4 전체 정리와 실습 · 과제 안내

요약 — Chapter 4

4.1 텍스트의 전처리 → 문자열을 분석 가능한 형태로

- 비정형 데이터 내의 오류 / 텍스트 문서의 변환 / 정제 · 정규화 · 토큰화 / 띄어쓰기 교정 / 맞춤법 교정 / 서브워드 토큰화

4.2 어휘 분석 → 의미를 가진 최소 단위 확정

- 형태소 분석 / 한국어 형태소 분석기 비교 / 품사와 품사 태그셋 / 품사 태깅 접근법(규칙 · 통계 · 딥러닝) / 활용 분야

4.3 구문 분석 → 단어 간 문법적 관계 규명

- 구문 분석 개요 / 구구조 문법 · 의존 문법 / 구문 중의성 / 구구조 구문 분석 / 의존 구문 분석(전이 · 그래프 기반) / 접근 방법의 장단점

4.4 의미 분석 → 문장이 뜻하는 바를 확정

- 단어와 단어 의미 중의성 / 중의성 해소(WSD) 기법 / 의미역 분석 / 의미 표현과 분포 가설

네 절을 관통하는 공통 흐름

- 규칙 기반 → 통계 · 확률 기반 → 딥러닝 기반. 그리고 하위 계층의 오류는 반드시 상위 계층으로 전파된다

참고 서적 및 자료

국내 서적

- 임희석, 『자연어처리 바이블 — 핵심이론 · 응용시스템 · 딥러닝』, 휴먼사이언스
- 전창욱 · 최태균 · 조중현 · 신성진, 『텐서플로 2와 머신러닝으로 시작하는 자연어 처리』, 위키북스
- 흡슨 레인 · 하네스 막스 하프케 · 콜 하워드, 『파이썬으로 배우는 자연어 처리 인 액션』, 제이펍
- 소함 고시(Sohom Ghosh) · 드와이트 거닝, 『예제로 배우는 자연어 처리 기초』, 에이콘

해외 교재

- Jurafsky & Martin, 『Speech and Language Processing』 (3rd ed. draft) — 온라인 무료 공개
- Manning & Schütze, 『Foundations of Statistical Natural Language Processing』

온라인 자료

- NLTK Book (nltk.org/book) · spaCy 101 (spacy.io/usage/spacy-101) · KoNLPy 문서 · Kiwi 문서
- Universal Dependencies (universaldependencies.org)

다음 주차 예고 — 단어 표현, 텍스트 분류, 그리고 텍스트 유사도

- 원-핫 인코딩과 그 한계 / BoW · N-gram / TF-IDF / 나이브 베이즈 · 로지스틱 회귀 / 자카드 · 코사인 · 유클리디언 · 맨해튼 유사도
- 이번 주차의 토큰이 다음 주차에는 벡터가 됩니다

Q&A — 질문 및 정리

1. 텍스트 전처리는 왜 모델링보다 먼저인가?

- 하위 계층의 오류가 상위 계층으로 전파되며, 입력 품질이 모델 성능의 상한을 결정하기 때문이다

2. 한국어 형태소 분석이 영어보다 어려운 이유는?

- 교착어라 어절 ≠ 단어이고, 조사 · 어미 결합으로 활용형이 폭발하며, 불규칙 활용과 띄어쓰기 오류가 겹치기 때문이다

3. 구구조 문법과 의존 문법 중 한국어에 적합한 것은?

- 어순이 자유롭고 조사가 문법 관계를 표시하므로 의존 문법이 더 자연스럽다

4. 문맥 임베딩은 WSD의 무엇을 해결했는가?

- 정적 임베딩이 한 단어에 한 벡터만 부여해 다의어를 구분하지 못하던 한계를 해결했다

5. 실습 노트북과 과제에 대한 질문